

Hoja de Trabajo Ejercicios SCD-ETL-ELT

Integrantes:

- Diego Alejandro Escobar Barahona
- Daniella Marissa Navarro

Repositorio de GitHub: https://github.com/Maxrissa/ETL_ELT.git

Ejercicio 1 — Sistema SCD para Recursos Humanos

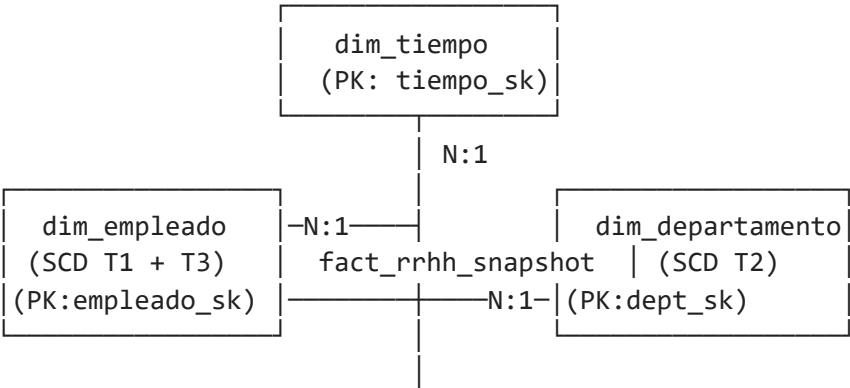
Motor: MySQL | **Schema:** Star Schema | **Enfoque:** Slowly Changing Dimensions (Tipo 1, 2 y 3)

Descripción General

Este notebook implementa un Data Warehouse de Recursos Humanos utilizando los tres tipos de SCD:

Tipo	Estrategia	Campo aplicado
SCD Tipo 1	Sobreescribe el valor (sin historia)	sede_actual en dim_empleado
SCD Tipo 2	Agrega nueva fila con fechas (historia completa)	dim_departamento
SCD Tipo 3	Columna adicional con valor anterior	salario_actual + salario_anterior en dim_empleado

Diagrama Star Schema



```
[fact_rrhh_snapshot]
Granularidad: 1 fila
por empleado/mes
```

Normalización

- **Fuente (Staging):** 3FN — sin redundancia, cada atributo depende de la PK
- **DWH Dimensiones:** Desnormalizado — todos los atributos en una fila para performance
- **Relaciones:** fact → dimensiones = **N:1** (muchas ventas, un empleado/depto/fecha)

1. Conexión a MySQL

```
In [4]: !pip install mysql-connector-python
```

Requirement already satisfied: mysql-connector-python in C:\Users\danie\miniconda3\lib\site-packages (9.7.0)

```
In [5]: import warnings
warnings.filterwarnings('ignore')

import mysql.connector
import pandas as pd
from IPython.display import display

# Configuración conexión

CONN_PARAMS = dict(
    host='127.0.0.1',
    user='Daniella',
    password='daniella123',
    port=3306,
    database='rrhh_dwh',
    charset='utf8mb4'
)

# Crear conexión

def get_conn():
    return mysql.connector.connect(
        host=CONN_PARAMS['host'],
        user=CONN_PARAMS['user'],
        password=CONN_PARAMS['password'],
        port=CONN_PARAMS['port'],
        database=CONN_PARAMS['database'],
        charset=CONN_PARAMS['charset']
    )

# Ejecutar consultas SELECT

def run_query(sql, params=None):

    conn = get_conn()
```

```

df = pd.read_sql(sql, conn, params=params)

conn.close()

return df

# Ejecutar CREATE INSERT UPDATE DELETE

def run_ddl(sql):

    conn = get_conn()

    cur = conn.cursor()

    for stmt in sql.split(';'):

        s = stmt.strip()

        if s:
            cur.execute(s)

    conn.commit()

    cur.close()
    conn.close()

# Crear base de datos

conn = mysql.connector.connect(
    host=CONN_PARAMS['host'],
    user=CONN_PARAMS['user'],
    password=CONN_PARAMS['password'],
    port=CONN_PARAMS['port'],
    charset='utf8mb4'
)

cur = conn.cursor()

cur.execute("DROP DATABASE IF EXISTS rrhh_dwh")

cur.execute("""
CREATE DATABASE rrhh_dwh
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci
""")

conn.commit()

cur.close()
conn.close()

print("Base de datos rrhh_dwh creada correctamente")

```

Base de datos rrhh_dwh creada correctamente

2. Tablas Staging

```
In [6]: # Tablas Staging (OLTP simulado - normalizadas en 3FN)
staging_sql = """
CREATE TABLE stg_empleados (
    empleado_id INT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE,
    fecha_ingreso DATE NOT NULL,
    departamento VARCHAR(80) NOT NULL,
    cargo VARCHAR(80) NOT NULL,
    salario DECIMAL(12,2) NOT NULL,
    sede VARCHAR(80) NOT NULL,
    activo TINYINT(1) DEFAULT 1,
    PRIMARY KEY (empleado_id)
);

CREATE TABLE stg_cambios_departamento (
    cambio_id INT NOT NULL AUTO_INCREMENT,
    empleado_id INT NOT NULL,
    dept_anterior VARCHAR(80),
    dept_nuevo VARCHAR(80) NOT NULL,
    fecha_cambio DATE NOT NULL,
    motivo VARCHAR(200),
    PRIMARY KEY (cambio_id)
);

CREATE TABLE stg_historial_salario (
    historial_id INT NOT NULL AUTO_INCREMENT,
    empleado_id INT NOT NULL,
    salario_nuevo DECIMAL(12,2) NOT NULL,
    fecha_cambio DATE NOT NULL,
    tipo_ajuste VARCHAR(50),
    PRIMARY KEY (historial_id)
)
"""
run_ddl(staging_sql)
print(" Tablas staging creadas (3FN - normalizadas).")
```

Tablas staging creadas (3FN - normalizadas).

3. Crear Dimensiones del DWH (Star Schema)

```
In [7]: dim_sql = """
-- DIMENSIÓN TIEMPO
CREATE TABLE dim_tiempo (
    tiempo_sk INT NOT NULL AUTO_INCREMENT,
    fecha_full DATE NOT NULL,
    anio SMALLINT NOT NULL,
    trimestre TINYINT NOT NULL,
    mes TINYINT NOT NULL,
    nombre_mes VARCHAR(20) NOT NULL,
```

```

    semana_anio TINYINT NOT NULL,
    dia_semana TINYINT NOT NULL,
    nombre_dia VARCHAR(20) NOT NULL,
    es_fin_semana TINYINT(1) NOT NULL DEFAULT 0,
    PRIMARY KEY (tiempo_sk),
    UNIQUE KEY uq_fecha (fecha_full)
);

-- DIMENSIÓN DEPARTAMENTO (SCD TIPO 2: filas con fecha_inicio/fin)
CREATE TABLE dim_departamento (
    dept_sk INT NOT NULL AUTO_INCREMENT,
    dept_nk INT NOT NULL,
    nombre_dept VARCHAR(80) NOT NULL,
    division VARCHAR(80),
    gerente VARCHAR(100),
    ubicacion VARCHAR(100),
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE,
    es_actual TINYINT(1) NOT NULL DEFAULT 1,
    PRIMARY KEY (dept_sk),
    INDEX idx_dept_nk (dept_nk),
    INDEX idx_dept_actual (dept_nk, es_actual)
);

-- DIMENSIÓN EMPLEADO (SCD TIPO 1: sede | SCD TIPO 3: salario)
CREATE TABLE dim_empleado (
    empleado_sk INT NOT NULL AUTO_INCREMENT,
    empleado_nk INT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    nombre_completo VARCHAR(200) GENERATED ALWAYS AS (CONCAT(nombre, ' ', apellido))
    email VARCHAR(150),
    fecha_ingreso DATE NOT NULL,
    cargo VARCHAR(80) NOT NULL,
    sede_actual VARCHAR(80) NOT NULL,
    salario_actual DECIMAL(12,2) NOT NULL,
    salario_anterior DECIMAL(12,2),
    fecha_ultimo_ajuste DATE,
    tipo_ultimo_ajuste VARCHAR(50),
    activo TINYINT(1) DEFAULT 1,
    fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP,
    fecha_actualizacion DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (empleado_sk),
    UNIQUE KEY uq_empleado_nk (empleado_nk)
)
""""
run_ddl(dim_sql)
print(" Dimensiones creadas.")
print(" → dim_tiempo : estática, PK surrogate")
print(" → dim_departamento : SCD Tipo 2 (filas históricas con fecha_inicio/fin)")
print(" → dim_empleado : SCD Tipo 1 (sede) + SCD Tipo 3 (salario)")

```

Dimensiones creadas.

- dim_tiempo : estática, PK surrogate
- dim_departamento : SCD Tipo 2 (filas históricas con fecha_inicio/fin)
- dim_empleado : SCD Tipo 1 (sede) + SCD Tipo 3 (salario)

```
In [8]: # Crear FACT TABLE
fact_sql = """
CREATE TABLE fact_rrhh_snapshot (
    snapshot_sk INT NOT NULL AUTO_INCREMENT,
    empleado_sk INT NOT NULL,
    dept_sk INT NOT NULL,
    tiempo_sk INT NOT NULL,
    salario_mes DECIMAL(12,2) NOT NULL,
    variacion_salarial_pct DECIMAL(8,4),
    dias_en_departamento INT,
    es_nuevo_ingreso TINYINT(1) DEFAULT 0,
    es_cambio_dept TINYINT(1) DEFAULT 0,
    es_aumento_salarial TINYINT(1) DEFAULT 0,
    PRIMARY KEY (snapshot_sk),
    CONSTRAINT fk_snap_empleado FOREIGN KEY (empleado_sk) REFERENCES dim_empleado(e
    CONSTRAINT fk_snap_dept FOREIGN KEY (dept_sk) REFERENCES dim_departamen
    CONSTRAINT fk_snap_tiempo FOREIGN KEY (tiempo_sk) REFERENCES dim_tiempo(tie
    INDEX idx_snap_empleado (empleado_sk),
    INDEX idx_snap_dept (dept_sk),
    INDEX idx_snap_tiempo (tiempo_sk)
)
"""
run_ddl(fact_sql)
print(" fact_rrhh_snapshot creada.")
print(" Tipo: Snapshot periódico | Granularidad: 1 fila por empleado por mes")
print(" Relaciones: N:1 → dim_empleado, dim_departamento, dim_tiempo")
```

fact_rrhh_snapshot creada.

Tipo: Snapshot periódico | Granularidad: 1 fila por empleado por mes

Relaciones: N:1 → dim_empleado, dim_departamento, dim_tiempo

4. Insertar Datos Staging

```
In [9]: insert_sql = """
INSERT INTO stg_empleados VALUES
(1,'Carlos','Mendoza','cmendoza@corp.com','2020-01-15','Tecnología','Dev Senior',75
(2,'Ana','García','agarcia@corp.com','2019-03-10','Ventas','Ejecutiva Sr',65000.00,
(3,'Roberto','López','rlopez@corp.com','2021-06-01','Finanzas','Analista',55000.00,
(4,'María','Sánchez','msanchez@corp.com','2018-09-20','RRHH','Coordinadora',60000.0
(5,'Javier','Torres','jtorres@corp.com','2022-02-14','Tecnología','Dev Junior',4500
"""
run_ddl(insert_sql)

insert_cambios = """
INSERT INTO stg_cambios_departamento (empleado_id,dept_anterior,dept_nuevo,fecha_ca
(1,'Tecnología','Arquitectura','2022-06-01','Promoción interna'),
(2,'Ventas','Marketing','2023-01-15','Reestructuración'),
(1,'Arquitectura','CTO Office','2023-09-01','Proyecto estratégico'),
(3,'Finanzas','Contraloría','2023-04-01','Reestructuración')
"""
run_ddl(insert_cambios)

insert_sal = """
INSERT INTO stg_historial_salario (empleado_id,salario_nuevo,fecha_cambio,tipo_ajus
```

```

(1,82000.00,'2022-06-01','PROMOCION'),
(1,90000.00,'2023-09-01','MERITO'),
(2,70000.00,'2023-01-15','PROMOCION'),
(3,60000.00,'2023-06-01','INFLACION'),
(4,67000.00,'2022-12-01','MERITO'),
(5,50000.00,'2023-03-01','MERITO')
"""

run_ddl(insert_sal)

df = run_query("SELECT * FROM stg_empleados")
print(f" Staging cargado. Empleados: {len(df)}")
display(df)

```

Staging cargado. Empleados: 5

	empleado_id	nombre	apellido	email	fecha_ingreso	departamento	
0	1	Carlos	Mendoza	cmendoza@corp.com	2020-01-15	Tecnología	De
1	2	Ana	García	agarcia@corp.com	2019-03-10	Ventas	Eje
2	3	Roberto	López	rlopez@corp.com	2021-06-01	Finanzas	
3	4	María	Sánchez	msanchez@corp.com	2018-09-20	RRHH	Coor
4	5	Javier	Torres	jtorres@corp.com	2022-02-14	Tecnología	Di

5. ETL — Carga de Dimensiones

```

In [10]: # Llenamos dim_tiempo con un procedimiento almacenado
proc_tiempo = """
DROP PROCEDURE IF EXISTS sp_fill_dim_tiempo
"""
run_ddl(proc_tiempo)

conn = get_conn()
cur = conn.cursor()
cur.execute("""
CREATE PROCEDURE sp_fill_dim_tiempo(IN p_start DATE, IN p_end DATE)
BEGIN
    DECLARE v_date DATE DEFAULT p_start;
    WHILE v_date <= p_end DO
        INSERT IGNORE INTO dim_tiempo (fecha_full,anio,trimestre,mes,nombre_mes,
            semana_anio,dia_semana,nombre_dia,es_fin_semana)
        VALUES (v_date,YEAR(v_date),QUARTER(v_date),MONTH(v_date),
            DATE_FORMAT(v_date,'%M'),WEEK(v_date,3),DAYOFWEEK(v_date),
            DATE_FORMAT(v_date,'%W'),IF(DAYOFWEEK(v_date) IN (1,7),1,0));
        SET v_date = DATE_ADD(v_date, INTERVAL 1 DAY);
    END WHILE;
END
""")
cur.callproc('sp_fill_dim_tiempo', ['2018-01-01', '2024-12-31'])

```

```

conn.commit()
conn.close()

cnt = run_query("SELECT COUNT(*) AS filas FROM dim_tiempo")
print(f" dim_tiempo cargada: {cnt['filas'][0]} días (2018-2024)")

```

dim_tiempo cargada: 2557 días (2018-2024)

```

In [11]: # SCD TIPO 2: Cargar dim_departamento
# 1) Estado inicial
carga_inicial = """
INSERT INTO dim_departamento (dept_nk,nombre_dept,division,gerente,ubicacion,fecha_
SELECT e.Empleado_id, e.departamento,
CASE
    WHEN e.departamento IN ('Tecnología','Arquitectura','CTO Office') THEN 'Tec
    WHEN e.departamento IN ('Ventas','Marketing') THEN 'Com
    WHEN e.departamento IN ('Finanzas','Contraloría') THEN 'Adm
    ELSE 'Corporativo'
END, NULL, e.sede, e.fecha_ingreso, NULL, 1
FROM stg_empleados e
"""
run_ddl(carga_inicial)

# 2) Cerrar registros anteriores (SCD T2: fecha_fin = día anterior al cambio)
cerrar = """
UPDATE dim_departamento dd
JOIN stg_cambios_departamento sc
ON sc.Empleado_id = dd.dept_nk AND dd.es_actual = 1 AND dd.nombre_dept = sc.dep
SET dd.fecha_fin = DATE_SUB(sc.fecha_cambio, INTERVAL 1 DAY),
    dd.es_actual = 0
"""
run_ddl(cerrar)

# 3) Insertar versión nueva
insertar_nueva = """
INSERT INTO dim_departamento (dept_nk,nombre_dept,division,gerente,ubicacion,fecha_
SELECT sc.Empleado_id, sc.dept_nuevo,
CASE
    WHEN sc.dept_nuevo IN ('Tecnología','Arquitectura','CTO Office') THEN 'Tec
    WHEN sc.dept_nuevo IN ('Ventas','Marketing') THEN 'Come
    WHEN sc.dept_nuevo IN ('Finanzas','Contraloría') THEN 'Admi
    ELSE 'Corporativo'
END, NULL, e.sede, sc.fecha_cambio, NULL, 1
FROM stg_cambios_departamento sc
JOIN stg_empleados e ON e.Empleado_id = sc.Empleado_id
ORDER BY sc.cambio_id
"""
run_ddl(insertar_nueva)

df = run_query("SELECT * FROM dim_departamento ORDER BY dept_nk, fecha_inicio")
print(f" dim_departamento cargada con SCD Tipo 2: {len(df)} filas")
display(df)

```

dim_departamento cargada con SCD Tipo 2: 9 filas

	dept_sk	dept_nk	nombre_dept	division	gerente	ubicacion	fecha_inicio	fecha_f
0	1	1	Tecnología	Tecnología y Sistemas	None	Ciudad de México	2020-01-15	2022-0
1	8	1	Arquitectura	Tecnología y Sistemas	None	Ciudad de México	2022-06-01	Noi
2	10	1	CTO Office	Tecnología y Sistemas	None	Ciudad de México	2023-09-01	Noi
3	2	2	Ventas	Comercial	None	Guadalajara	2019-03-10	2023-0
4	9	2	Marketing	Comercial	None	Guadalajara	2023-01-15	Noi
5	3	3	Finanzas	Administración	None	Ciudad de México	2021-06-01	2023-0
6	11	3	Contraloría	Administración	None	Ciudad de México	2023-04-01	Noi
7	4	4	RRHH	Corporativo	None	Monterrey	2018-09-20	Noi
8	5	5	Tecnología	Tecnología y Sistemas	None	Guadalajara	2022-02-14	Noi

```
In [12]: # SCD TIPO 1 (sede) + SCD TIPO 3 (salario): Cargar dim_empleado
cargar_empleado = """
INSERT INTO dim_empleado (empleado_nk,nombre,apellido,email,fecha_ingreso,cargo,
sede_actual,salario_actual,salario_anterior,fecha_ultimo_ajuste,tipo_ultimo_aju
SELECT
    e.empleado_id,
    TRIM(e.nombre), TRIM(e.apellido), LOWER(TRIM(e.email)),
    e.fecha_ingreso, TRIM(e.cargo),
    TRIM(e.sede),
    COALESCE(ultimo.salario_nuevo, e.salario),
    CASE WHEN ultimo.salario_nuevo IS NOT NULL THEN e.salario ELSE NULL END,
    ultimo.fecha_cambio, ultimo.tipo_ajuste, e.activo
FROM stg_empleados e
LEFT JOIN (
    SELECT h1.* FROM stg_historial_salario h1
    WHERE h1.historial_id = (
        SELECT MAX(h2.historial_id) FROM stg_historial_salario h2
        WHERE h2.empleado_id = h1.empleado_id)
) ultimo ON ultimo.empleado_id = e.empleado_id
"""
run_ddl(cargar_empleado)

df = run_query("SELECT empleado_nk,nombre_completo,cargo,sede_actual,salario_anteri
print("dim_empleado cargada")
print(" SCD T1 → sede_actual (sobreescribe sin historia)")
print(" SCD T3 → salario_anterior + salario_actual")
display(df)
```

dim_empleado cargada

SCD T1 → sede_actual (sobreescribe sin historia)

SCD T3 → salario_anterior + salario_actual

	empleado_nk	nombre_completo	cargo	sede_actual	salario_anterior	salario_actual
0	1	Carlos Mendoza	Dev Senior	Ciudad de México	75000.0	90000.0
1	2	Ana García	Ejecutiva Sr	Guadalajara	65000.0	70000.0
2	3	Roberto López	Analista	Ciudad de México	55000.0	60000.0
3	4	María Sánchez	Coordinadora	Monterrey	60000.0	67000.0
4	5	Javier Torres	Dev Junior	Guadalajara	45000.0	50000.0



```
In [13]: # SIMULAR SCD TIPO 1: Cambio de sede de Carlos Mendoza
run_ddl("UPDATE dim_empleado SET sede_actual = 'Monterrey' WHERE empleado_nk = 1")
df = run_query("SELECT nombre_completo, sede_actual FROM dim_empleado WHERE emplead
print(" SCD Tipo 1 aplicado – Carlos Mendoza cambió de sede:")
display(df)
print(" Historia de sede PERDIDA (eso es SCD T1: sobreescribe)")
```

SCD Tipo 1 aplicado – Carlos Mendoza cambió de sede:

	nombre_completo	sede_actual
0	Carlos Mendoza	Monterrey

Historia de sede PERDIDA (eso es SCD T1: sobreescribe)

6. Carga Fact Table y Análisis

```
In [18]: # Cargar snapshot mensual
snap_sql = """
INSERT INTO fact_rrhh_snapshot (
    empleado_sk, dept_sk, tiempo_sk, salario_mes,
    variacion_salarial_pct, dias_en_departamento,
    es_nuevo_ingreso, es_cambio_dept, es_aumento_salarial)
SELECT
    de.empleado_sk, dd.dept_sk, dt.tiempo_sk,
    de.salario_actual,
    CASE WHEN de.salario_anterior IS NOT NULL AND de.salario_anterior > 0
        THEN ROUND(((de.salario_actual - de.salario_anterior)/de.salario_anterior)
        ELSE NULL END,
    DATEDIFF(COALESCE(dd.fecha_fin, CURDATE()), dd.fecha_inicio),
    IF(YEAR(se.fecha_ingreso)=YEAR(dd.fecha_inicio) AND MONTH(se.fecha_ingreso)=MON
    IF(dd.fecha_inicio > se.fecha_ingreso,1,0),
    IF(de.salario_anterior IS NOT NULL,1,0)
FROM dim_empleado de
JOIN stg_empleados se    ON se.empleado_id = de.empleado_nk
JOIN dim_departamento dd ON dd.dept_nk = de.empleado_nk AND dd.es_actual = 1
JOIN dim_tiempo dt      ON dt.fecha_full = LAST_DAY(CURDATE())
```

```

"""
run_ddl(snap_sql)

df = run_query("""
SELECT de.nombre_completo, dd.nombre_dept, dd.division,
      dt.fecha_full, fs.salario_mes, fs.variacion_salarial_pct,
      IF(fs.es_aumento_salarial=1,'SÍ','NO') AS aumento,
      IF(fs.es_cambio_dept=1,'SÍ','NO') AS cambio_dept
FROM fact_rrhh_snapshot fs
JOIN dim_empleado de ON de.empleado_sk=fs.empleado_sk
JOIN dim_departamento dd ON dd.dept_sk=fs.dept_sk
JOIN dim_tiempo dt ON dt.tiempo_sk=fs.tiempo_sk
""")
print("fact_rrhh_snapshot cargada")
display(df)

```

fact_rrhh_snapshot cargada

nombre_completo	nombre_dept	division	fecha_full	salario_mes	variacion_salarial_pct	au
-----------------	-------------	----------	------------	-------------	------------------------	----



7. Análisis — Historia Completa por Empleado (SCD T2)

```

In [19]: df_hist = run_query("""
SELECT de.nombre_completo AS empleado, dd.nombre_dept AS departamento,
      dd.division, dd.fecha_inicio, COALESCE(dd.fecha_fin,'9999-12-31') AS fecha_f
      dd.es_actual,
      DATEDIFF(COALESCE(dd.fecha_fin, CURDATE()), dd.fecha_inicio) AS dias_en_dept
FROM dim_departamento dd
JOIN dim_empleado de ON de.empleado_nk = dd.dept_nk
ORDER BY de.empleado_nk, dd.fecha_inicio
""")
print(" Historial completo de departamentos (SCD Tipo 2)")
display(df_hist)

```

Historial completo de departamentos (SCD Tipo 2)

	empleado	departamento	division	fecha_inicio	fecha_fin	es_actual	dias_en_dept
0	Carlos Mendoza	Tecnología	Tecnología y Sistemas	2020-01-15	2022-05-31	0	867
1	Carlos Mendoza	Arquitectura	Tecnología y Sistemas	2022-06-01	9999-12-31	1	1454
2	Carlos Mendoza	CTO Office	Tecnología y Sistemas	2023-09-01	9999-12-31	1	997
3	Ana García	Ventas	Comercial	2019-03-10	2023-01-14	0	1406
4	Ana García	Marketing	Comercial	2023-01-15	9999-12-31	1	1226
5	Roberto López	Finanzas	Administración	2021-06-01	2023-03-31	0	668
6	Roberto López	Contraloría	Administración	2023-04-01	9999-12-31	1	1150
7	María Sánchez	RRHH	Corporativo	2018-09-20	9999-12-31	1	2804
8	Javier Torres	Tecnología	Tecnología y Sistemas	2022-02-14	9999-12-31	1	1561

8. Evolución Salarial (SCD Tipo 3)

```
In [16]: df_sal = run_query("""
SELECT nombre_completo, cargo, sede_actual AS sede_actual_SCD_T1,
       salario_anterior AS salario_previo_SCD_T3, salario_actual AS salario_vigente
       CASE WHEN salario_anterior IS NOT NULL
            THEN CONCAT(ROUND(((salario_actual-salario_anterior)/salario_anterior)*
            ELSE 'Sin cambio' END AS variacion,
       fecha_ultimo_ajuste, tipo_ultimo_ajuste
FROM dim_empleado
ORDER BY salario_actual DESC
""")
print(" Evolución salarial – SCD Tipo 3 (actual vs anterior)")
display(df_sal)
```

Evolución salarial – SCD Tipo 3 (actual vs anterior)

	nombre_completo	cargo	sede_actual_SCD_T1	salario_previo_SCD_T3	salario_vigent
0	Carlos Mendoza	Dev Senior	Monterrey	75000.0	
1	Ana García	Ejecutiva Sr	Guadalajara	65000.0	
2	María Sánchez	Coordinadora	Monterrey	60000.0	
3	Roberto López	Analista	Ciudad de México	55000.0	
4	Javier Torres	Dev Junior	Guadalajara	45000.0	

9. Comparación entre Tipos SCD

```
In [17]: df_comp = run_query("""
SELECT 'SCD Tipo 1' AS tipo_scd, 'sede_actual en dim_empleado' AS ubicacion,
      'Sobreescribe sin historia. Solo valor actual visible.' AS descripcion,
      COUNT(*) AS registros_afectados
FROM dim_empleado WHERE sede_actual IS NOT NULL
UNION ALL
SELECT 'SCD Tipo 2','dim_departamento (múltiples filas)',
      'Nueva fila por cada cambio. Historia completa con fecha_inicio/fin.',
      COUNT(*) FROM dim_departamento WHERE es_actual = 0
UNION ALL
SELECT 'SCD Tipo 3','salario_actual + salario_anterior en dim_empleado',
      'Dos columnas: valor actual y valor inmediato anterior. Solo 1 nivel de hist
      COUNT(*) FROM dim_empleado WHERE salario_anterior IS NOT NULL
      """)
print("Comparación entre tipos SCD")
display(df_comp)
```

Comparación entre tipos SCD

	tipo_scd	ubicacion	descripcion	registros_afectados
0	SCD Tipo 1	sede_actual en dim_empleado	Sobreescribe sin historia. Solo valor actual v...	5
1	SCD Tipo 2	dim_departamento (múltiples filas)	Nueva fila por cada cambio. Historia completa ...	3
2	SCD Tipo 3	salario_actual + salario_anterior en dim_empleado	Dos columnas: valor actual y valor inmediato a...	5

10. Resumen

Elemento	Detalle
Schema	Star Schema — fact apunta directamente a dims
Tablas factuales	fact_rrhh_snapshot (snapshot periódico mensual)

Elemento	Detalle
Tablas dimensionales	<code>dim_tiempo</code> , <code>dim_departamento</code> (SCD T2), <code>dim_empleado</code> (SCD T1+T3)
Normalización fuente	3FN (staging)
Normalización DWH	Desnormalizado (dims planas)
Relaciones	fact → dims = N:1
SCD T1	<code>sede_actual</code> sobrescrita sin historia
SCD T2	<code>dim_departamento</code> con <code>fecha_inicio/fin</code> y <code>es_actual</code>
SCD T3	<code>salario_actual</code> + <code>salario_anterior</code> en misma fila

ETL para Star Schema - empresa_etl

Flujo: EXTRACT -> TRANSFORM -> LOAD -> Star Schema + KPIs

Que es ETL?

Fase	Descripcion
Extract	Carga los datos crudos desde la fuente
Transform	Limpia y valida los datos
Load	Carga los datos limpios al modelo final

Que es un Star Schema?

Un esquema en estrella organiza los datos en:

- Tablas de Dimensiones (DIM): describen el quien, que, donde
- Tabla de Hechos (FACT): almacena los eventos medibles como ventas

```
      dim_cliente
      |
dim_empleado -- fact_ventas -- dim_producto
```

Conexion a MySQL

```
In [1]: %load_ext sql

usuario      = 'Daniella'
contrasena   = 'daniella123'
host         = '127.0.0.1'
puerto      = '3306'

%sql mysql+pymysql://Daniella:daniella123@127.0.0.1:3306/
```

Crear la base de datos empresa_etl

```
In [2]: %%sql
DROP DATABASE IF EXISTS empresa_etl;

CREATE DATABASE empresa_etl
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
12 rows affected.
1 rows affected.

Out[2]: []

In [3]: %%sql
USE empresa_etl;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.

Out[3]: []

```

1. EXTRACT - Tablas Staging

En esta fase los datos se depositan en tablas staging sin ninguna transformacion. Estos datos pueden tener NULLs y duplicados, lo cual es normal en datos crudos.

Problema	Tabla
cliente_id = 4 duplicado	stg_clientes
nombre = NULL	stg_clientes, stg_empleados
producto = NULL	stg_productos

```

In [4]: %%sql
DROP TABLE IF EXISTS stg_clientes;
CREATE TABLE stg_clientes (
    cliente_id INT,
    nombre      VARCHAR(100),
    ciudad      VARCHAR(100),
    edad        INT
);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.

Out[4]: []

```

```

In [5]: %%sql
INSERT INTO stg_clientes VALUES
(1, 'Carlos', 'San Salvador', 25),
(2, 'Ana',    'Santa Ana',    30),
(3, NULL,    'San Miguel',    22),
(4, 'Pedro', 'La Libertad', 40),
(4, 'Pedro', 'La Libertad', 40);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

Out[5]: []

```

```
In [6]: %%sql
DROP TABLE IF EXISTS stg_productos;
CREATE TABLE stg_productos (
    producto_id INT,
    producto     VARCHAR(100),
    categoria    VARCHAR(100),
    precio       DECIMAL(10,2)
);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
```

Out[6]: []

```
In [7]: %%sql
INSERT INTO stg_productos VALUES
(1, 'Laptop', 'Tecnologia', 1200.00),
(2, 'Mouse',  'Tecnologia',  25.00),
(3, 'Silla',  'Muebles',    150.00),
(4, NULL,    'Tecnologia',  75.00);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[7]: []

```
In [8]: %%sql
DROP TABLE IF EXISTS stg_empleados;
CREATE TABLE stg_empleados (
    empleado_id INT,
    nombre       VARCHAR(100),
    departamento VARCHAR(100)
);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
```

Out[8]: []

```
In [9]: %%sql
INSERT INTO stg_empleados VALUES
(1, 'Maria', 'Ventas'),
(2, 'Jose',  'Ventas'),
(3, NULL,    'Marketing');

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

Out[9]: []

```
In [10]: %%sql
DROP TABLE IF EXISTS stg_ventas;
CREATE TABLE stg_ventas (
    venta_id     INT,
    cliente_id   INT,
    producto_id  INT,
```

```

    empleado_id INT,
    cantidad     INT
);

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.

```

Out[10]: []

```

In [11]: %%sql
INSERT INTO stg_ventas VALUES
(1, 1, 1, 1, 2),
(2, 2, 2, 1, 5),
(3, 3, 3, 2, 1),
(4, 4, 1, 2, 1),
(5, 1, 2, 1, 3);

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

```

Out[11]: []

Ver datos crudos

```

In [12]: %%sql
SELECT 'stg_clientes' AS tabla, COUNT(*) AS filas FROM stg_clientes
UNION ALL
SELECT 'stg_productos', COUNT(*) FROM stg_productos
UNION ALL
SELECT 'stg_empleados', COUNT(*) FROM stg_empleados
UNION ALL
SELECT 'stg_ventas',    COUNT(*) FROM stg_ventas;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.

```

Out[12]:

tabla	filas
stg_clientes	5
stg_productos	4
stg_empleados	3
stg_ventas	5

```

In [13]: %%sql
SELECT * FROM stg_clientes;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

```

Out[13]: **cliente_id nombre ciudad edad**

1	Carlos	San Salvador	25
2	Ana	Santa Ana	30
3	None	San Miguel	22
4	Pedro	La Libertad	40
4	Pedro	La Libertad	40

In [14]: **%%sql**
SELECT * FROM stg_productos;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.

Out[14]: **producto_id producto categoria precio**

1	Laptop	Tecnologia	1200.00
2	Mouse	Tecnologia	25.00
3	Silla	Muebles	150.00
4	None	Tecnologia	75.00

In [15]: **%%sql**
SELECT * FROM stg_empleados;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.

Out[15]: **empleado_id nombre departamento**

1	Maria	Ventas
2	Jose	Ventas
3	None	Marketing

In [16]: **%%sql**
SELECT * FROM stg_ventas;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

Out[16]: **venta_id cliente_id producto_id empleado_id cantidad**

1	1	1	1	2
2	2	2	1	5
3	3	3	2	1
4	4	1	2	1
5	1	2	1	3

2. TRANSFORM - Limpieza de datos

Se aplican reglas para corregir los datos antes de cargarlos al modelo final.

Problema	Solucion SQL
Nombres NULL en clientes/empleados	COALESCE(nombre, 'Sin Nombre')
Nombre de producto NULL	COALESCE(producto, 'Producto Desconocido')
Filas duplicadas	SELECT DISTINCT
Campo total faltante	cantidad x precio via JOIN

```
In [17]: %%sql
DROP TABLE IF EXISTS clientes_clean;
CREATE TABLE clientes_clean AS
SELECT DISTINCT
    cliente_id,
    COALESCE(nombre, 'Sin Nombre') AS nombre,
    ciudad,
    edad
FROM stg_clientes;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
4 rows affected.
```

Out[17]: []

```
In [18]: %%sql
DROP TABLE IF EXISTS productos_clean;
CREATE TABLE productos_clean AS
SELECT DISTINCT
    producto_id,
    COALESCE(producto, 'Producto Desconocido') AS producto,
    categoria,
    precio
FROM stg_productos;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
4 rows affected.
```

Out[18]: []

```
In [19]: %%sql
DROP TABLE IF EXISTS empleados_clean;
CREATE TABLE empleados_clean AS
SELECT DISTINCT
    empleado_id,
    COALESCE(nombre, 'Sin Nombre') AS nombre,
```

```
departamento
FROM stg_empleados;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
3 rows affected.
```

Out[19]: []

```
In [20]: %%sql
DROP TABLE IF EXISTS ventas_transformadas;
CREATE TABLE ventas_transformadas AS
SELECT
    v.venta_id,
    v.cliente_id,
    v.producto_id,
    v.empleado_id,
    v.cantidad,
    p.precio,
    (v.cantidad * p.precio) AS total
FROM stg_ventas v
JOIN productos_clean p ON v.producto_id = p.producto_id;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
5 rows affected.
```

Out[20]: []

Verificar resultados de la limpieza

```
In [21]: %%sql
-- Antes: 5 filas con duplicado y NULL | Despues: 4 filas limpias
SELECT * FROM clientes_clean;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[21]:

cliente_id	nombre	ciudad	edad
------------	--------	--------	------

1	Carlos	San Salvador	25
2	Ana	Santa Ana	30
3	Sin Nombre	San Miguel	22
4	Pedro	La Libertad	40

```
In [22]: %%sql
SELECT * FROM productos_clean;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

```
Out[22]:
```

	producto_id	producto	categoria	precio
	1	Laptop	Tecnologia	1200.00
	2	Mouse	Tecnologia	25.00
	3	Silla	Muebles	150.00
	4	Producto Desconocido	Tecnologia	75.00

```
In [23]: %%sql
SELECT * FROM empleados_clean;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

```
Out[23]:
```

	empleado_id	nombre	departamento
	1	Maria	Ventas
	2	Jose	Ventas
	3	Sin Nombre	Marketing

```
In [24]: %%sql
SELECT * FROM ventas_transformadas;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

```
Out[24]:
```

	venta_id	cliente_id	producto_id	empleado_id	cantidad	precio	total
	1	1	1	1	2	1200.00	2400.00
	2	2	2	1	5	25.00	125.00
	3	3	3	2	1	150.00	150.00
	4	4	1	2	1	1200.00	1200.00
	5	1	2	1	3	25.00	75.00

3. LOAD - Dimensiones

Las tablas de dimension son el contexto descriptivo del Star Schema. Cada una tiene una clave primaria (PK) que la vincula a la tabla de hechos.

dim_cliente

```
In [25]: %%sql
DROP TABLE IF EXISTS dim_cliente;
CREATE TABLE dim_cliente (
    cliente_id INT PRIMARY KEY,
```

```

    nombre    VARCHAR(100),
    ciudad    VARCHAR(100),
    edad      INT
);

INSERT INTO dim_cliente
SELECT * FROM clientes_clean;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
4 rows affected.

```

Out[25]: []

```

In [26]: %%sql
SELECT * FROM dim_cliente;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.

```

Out[26]:

cliente_id	nombre	ciudad	edad
1	Carlos	San Salvador	25
2	Ana	Santa Ana	30
3	Sin Nombre	San Miguel	22
4	Pedro	La Libertad	40

dim_producto

```

In [27]: %%sql
DROP TABLE IF EXISTS dim_producto;
CREATE TABLE dim_producto (
    producto_id INT PRIMARY KEY,
    producto    VARCHAR(100),
    categoria    VARCHAR(100),
    precio      DECIMAL(10,2)
);

```

```

INSERT INTO dim_producto
SELECT * FROM productos_clean;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
4 rows affected.

```

Out[27]: []

```

In [28]: %%sql
SELECT * FROM dim_producto;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.

```

```
Out[28]:
```

	producto_id	producto	categoria	precio
	1	Laptop	Tecnologia	1200.00
	2	Mouse	Tecnologia	25.00
	3	Silla	Muebles	150.00
	4	Producto Desconocido	Tecnologia	75.00

dim_empleado

```
In [29]: %%sql
DROP TABLE IF EXISTS dim_empleado;
CREATE TABLE dim_empleado (
    empleado_id INT PRIMARY KEY,
    nombre      VARCHAR(100),
    departamento VARCHAR(100)
);

INSERT INTO dim_empleado
SELECT * FROM empleados_clean;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
3 rows affected.
```

```
Out[29]: []
```

```
In [30]: %%sql
SELECT * FROM dim_empleado;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

```
Out[30]:
```

	empleado_id	nombre	departamento
	1	Maria	Ventas
	2	Jose	Ventas
	3	Sin Nombre	Marketing

4. LOAD - Fact Table

La tabla de hechos es el centro del Star Schema. Contiene las metricas medibles (cantidad, precio, total) y las claves foraneas (FK) que apuntan a cada dimension.

```
In [31]: %%sql
DROP TABLE IF EXISTS fact_ventas;
CREATE TABLE fact_ventas (
    venta_id INT PRIMARY KEY,
```

```

    cliente_id INT,
    producto_id INT,
    empleado_id INT,
    cantidad INT,
    precio DECIMAL(10,2),
    total DECIMAL(10,2),

    FOREIGN KEY (cliente_id) REFERENCES dim_cliente(cliente_id),
    FOREIGN KEY (producto_id) REFERENCES dim_producto(producto_id),
    FOREIGN KEY (empleado_id) REFERENCES dim_empleado(empleado_id)
);

INSERT INTO fact_ventas
SELECT venta_id, cliente_id, producto_id, empleado_id, cantidad, precio, total
FROM ventas_transformadas;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
5 rows affected.

```

Out[31]: []

```

In [32]: %%sql
SELECT * FROM fact_ventas;

```

```

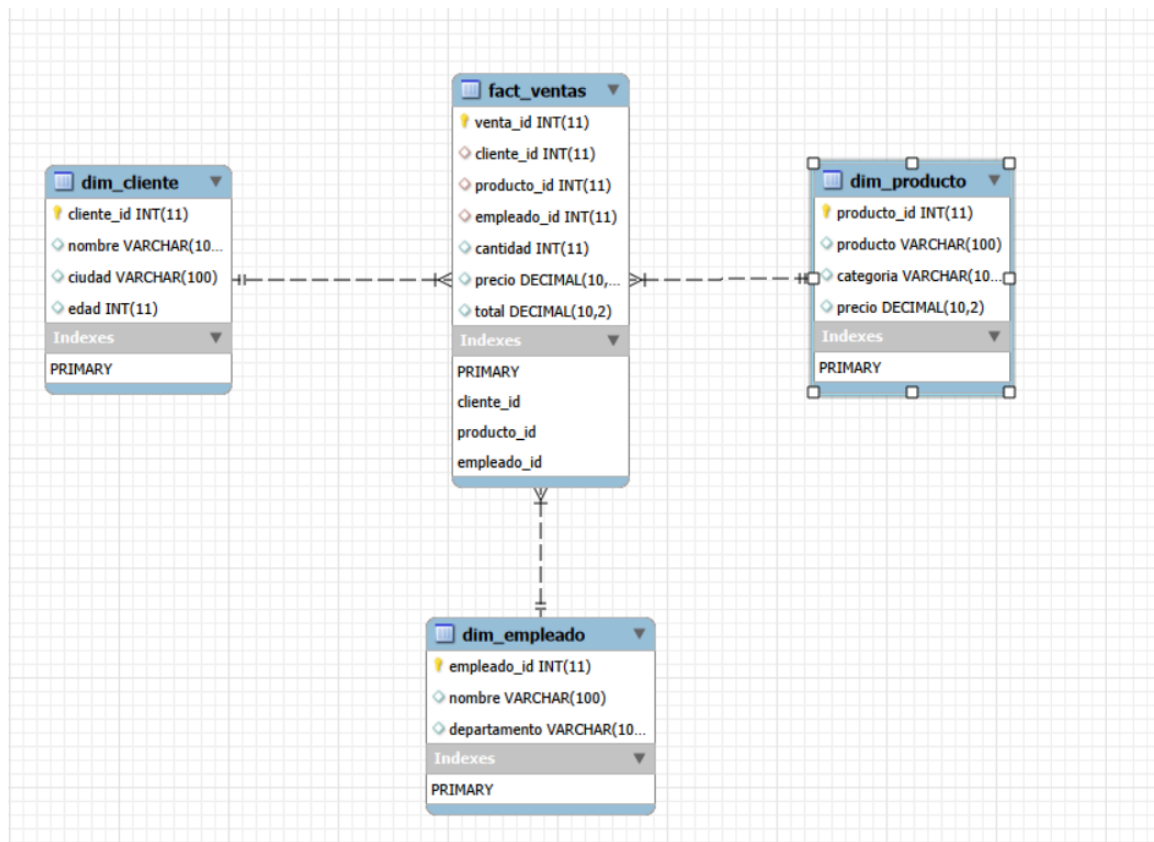
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

```

Out[32]:

venta_id	cliente_id	producto_id	empleado_id	cantidad	precio	total
1	1	1	1	2	1200.00	2400.00
2	2	2	1	5	25.00	125.00
3	3	3	2	1	150.00	150.00
4	4	1	2	1	1200.00	1200.00
5	1	2	1	3	25.00	75.00

5. Diagrama del Star Schema



En el centro — fact_ventas: esta es la tabla más importante. Guarda cada venta que ocurrió, cuánto se vendió, el precio y el total. No repite información de clientes ni productos, solo guarda los IDs para conectarse con las otras tablas.

En los brazos — las dimensiones: son dim_cliente, dim_producto y dim_empleado. Cada una describe el contexto de la venta, quién compró, qué se vendió y quién lo vendió. Todo el detalle vive directamente dentro de cada tabla: la ciudad está en dim_cliente, la categoría está en dim_producto, el departamento está en dim_empleado. y se conectan a través de las Foreign Keys (FK). La fact_ventas tiene cliente_id, producto_id y empleado_id, y con esos IDs puede ir a buscar el detalle a cada dimensión.

6. KPIs - Consultas analíticas

KPI 1 - Ingreso total y promedio por venta

```
In [33]: %%sql
SELECT
    COUNT(*)      AS num_ventas,
    SUM(total)    AS ingreso_total,
    AVG(total)    AS promedio_venta
FROM fact_ventas;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
1 rows affected.
```

Out[33]: **num_ventas ingreso_total promedio_venta**

5	3950.00	790.000000
---	---------	------------

KPI 2 - Ventas por ciudad

```
In [34]: %%sql
SELECT
    c.ciudad,
    SUM(f.total) AS ventas
FROM fact_ventas f
JOIN dim_cliente c ON f.cliente_id = c.cliente_id
GROUP BY c.ciudad
ORDER BY ventas DESC;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[34]: **ciudad ventas**

San Salvador	2475.00
La Libertad	1200.00
San Miguel	150.00
Santa Ana	125.00

KPI 3 - Top clientes por monto comprado

```
In [35]: %%sql
SELECT
    c.nombre AS cliente,
    SUM(f.total) AS total_comprado
FROM fact_ventas f
JOIN dim_cliente c ON f.cliente_id = c.cliente_id
GROUP BY c.nombre
ORDER BY total_comprado DESC;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[35]: **cliente total_comprado**

Carlos	2475.00
Pedro	1200.00
Sin Nombre	150.00
Ana	125.00

KPI 4 - Ingresos por categoria de producto

```
In [36]: %%sql
SELECT
    p.categoria,
    SUM(f.total) AS ingresos
FROM fact_ventas f
JOIN dim_producto p ON f.producto_id = p.producto_id
GROUP BY p.categoria
ORDER BY ingresos DESC;
```

```
* mysql+pymysql://Daniella:**@127.0.0.1:3306/
2 rows affected.
```

```
Out[36]: categoria  ingresos
Tecnologia      3800.00
Muebles         150.00
```

KPI 5 - Rendimiento de empleados

```
In [37]: %%sql
SELECT
    e.nombre      AS empleado,
    SUM(f.total)  AS ventas_generadas
FROM fact_ventas f
JOIN dim_empleado e ON f.empleado_id = e.empleado_id
GROUP BY e.nombre
ORDER BY ventas_generadas DESC;
```

```
* mysql+pymysql://Daniella:**@127.0.0.1:3306/
2 rows affected.
```

```
Out[37]: empleado  ventas_generadas
Maria          2600.00
Jose           1350.00
```

7. Resumen del proceso ETL

Paso	Fase	Accion
1	Extract	Datos crudos en tablas staging en MySQL
2	Transform	COALESCE para NULLs, DISTINCT para duplicados, JOIN para calcular total
3	Load DIM	dim_cliente, dim_producto, dim_empleado con claves primarias
4	Load FACT	fact_ventas con FK y metricas de negocio

Paso	Fase	Accion
5	Star Schema	Diagrama visual del modelo dimensional
6	KPIs	5 consultas analiticas con graficos

Problemas detectados y resueltos

Problema	Tabla	Solucion
Fila duplicada Pedro id=4	stg_clientes	SELECT DISTINCT
nombre = NULL	stg_clientes, stg_empleados	COALESCE(nombre, 'Sin Nombre')
producto = NULL	stg_productos	COALESCE(producto, 'Producto Desconocido')
Campo total no existia	stg_ventas	cantidad x precio via JOIN

ELT para Snowflake Schema - empresa_elt

Flujo: EXTRACT -> LOAD -> TRANSFORM -> Snowflake Schema + KPIs

¿Qué es ELT?

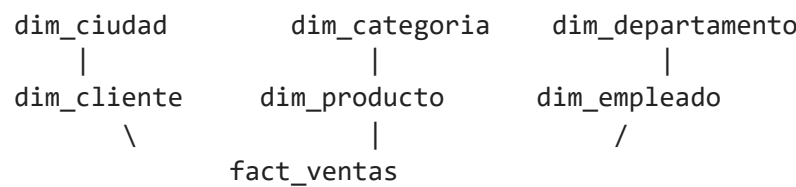
Fase	Descripción
Extract	Carga los datos crudos desde la fuente
Load	Carga los datos sin limpiar directamente a la base de datos destino
Transform	Limpia y normaliza los datos dentro de la base de datos

Diferencia con ETL: En ELT los datos se cargan primero y se transforman después, aprovechando el motor de la base de datos para las transformaciones.

¿Qué es un Snowflake Schema?

Un esquema en copo de nieve es una extensión del Star Schema donde las dimensiones están normalizadas en sub-dimensiones:

- dim_cliente → depende de → dim_ciudad
- dim_producto → depende de → dim_categoria
- dim_empleado → depende de → dim_departamento
- fact_ventas → centro que une todas las dimensiones



Característica	Star Schema	Snowflake Schema
Dimensiones normalizadas	No	Sí
Tablas	Menos	Más
Redundancia	Alta	Baja
Consultas	Más simples	Requieren más JOINS

Conexión a MySQL

```
In [1]: %load_ext sql

usuario      = 'Daniella'
contrasena   = 'daniella123'
host         = '127.0.0.1'
puerto      = '3306'

%sql mysql+pymysql://Daniella:daniella123@127.0.0.1:3306/
```

Crear la base de datos empresa_elt

```
In [2]: %%sql
DROP DATABASE IF EXISTS empresa_elt;

CREATE DATABASE empresa_elt
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
15 rows affected.
1 rows affected.
```

Out[2]: []

```
In [3]: %%sql
USE empresa_elt;
SET SQL_SAFE_UPDATES = 0;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
```

Out[3]: []

1. EXTRACT - Tablas Staging

En ELT los datos se **extraen tal como están** desde la fuente hacia tablas staging. No se aplica ninguna transformación aquí — los datos pueden tener NULLs y duplicados.

Problema	Tabla
cliente_id = 4 duplicado	stg_clientes
nombre = NULL	stg_clientes, stg_empleados
producto = NULL	stg_productos

```
In [4]: %%sql
CREATE TABLE stg_clientes (
    cliente_id INT,
    nombre      VARCHAR(100),
    ciudad      VARCHAR(100),
    edad        INT
);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
```

Out[4]: []

```
In [5]: %%sql
INSERT INTO stg_clientes VALUES
(1, 'Carlos', 'San Salvador', 25),
(2, 'Ana',    'Santa Ana',    30),
(3, NULL,    'San Miguel',    22),
(4, 'Pedro',  'La Libertad',  40),
(4, 'Pedro',  'La Libertad',  40);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

Out[5]: []

```
In [6]: %%sql
CREATE TABLE stg_productos (
    producto_id INT,
    producto     VARCHAR(100),
    categoria    VARCHAR(100),
    precio       DECIMAL(10,2)
);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
```

Out[6]: []

```
In [7]: %%sql
INSERT INTO stg_productos VALUES
(1, 'Laptop', 'Tecnologia', 1200.00),
(2, 'Mouse',  'Tecnologia',  25.00),
(3, 'Silla',  'Muebles',     150.00),
(4, NULL,    'Tecnologia',   75.00);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[7]: []

```
In [8]: %%sql
CREATE TABLE stg_empleados (
    empleado_id INT,
    nombre       VARCHAR(100),
    departamento VARCHAR(100)
);
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
```

Out[8]: []

```
In [9]: %%sql
INSERT INTO stg_empleados VALUES
(1, 'Maria', 'Ventas'),
(2, 'Jose', 'Ventas'),
(3, NULL, 'Marketing');
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

Out[9]: []

```
In [10]: %%sql
CREATE TABLE stg_ventas (
    venta_id INT,
    cliente_id INT,
    producto_id INT,
    empleado_id INT,
    cantidad INT
);
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
```

Out[10]: []

```
In [11]: %%sql
INSERT INTO stg_ventas VALUES
(1, 1, 1, 1, 2),
(2, 2, 2, 1, 5),
(3, 3, 3, 2, 1),
(4, 4, 1, 2, 1),
(5, 1, 2, 1, 3);
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

Out[11]: []

Ver datos crudos en staging

```
In [12]: %%sql
SELECT 'stg_clientes' AS tabla, COUNT(*) AS filas FROM stg_clientes
UNION ALL
SELECT 'stg_productos', COUNT(*) FROM stg_productos
UNION ALL
SELECT 'stg_empleados', COUNT(*) FROM stg_empleados
UNION ALL
SELECT 'stg_ventas', COUNT(*) FROM stg_ventas;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[12]:

tabla	filas
stg_clientes	5
stg_productos	4
stg_empleados	3
stg_ventas	5

In [13]:

```
%%sql
SELECT * FROM stg_clientes;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

Out[13]:

cliente_id	nombre	ciudad	edad
1	Carlos	San Salvador	25
2	Ana	Santa Ana	30
3	None	San Miguel	22
4	Pedro	La Libertad	40
4	Pedro	La Libertad	40

In [14]:

```
%%sql
SELECT * FROM stg_productos;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[14]:

producto_id	producto	categoria	precio
1	Laptop	Tecnologia	1200.00
2	Mouse	Tecnologia	25.00
3	Silla	Muebles	150.00
4	None	Tecnologia	75.00

In [15]:

```
%%sql
SELECT * FROM stg_empleados;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

Out[15]:

empleado_id	nombre	departamento
1	Maria	Ventas
2	Jose	Ventas
3	None	Marketing

```
In [16]: %%sql
SELECT * FROM stg_ventas;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

```
Out[16]: venta_id cliente_id producto_id empleado_id cantidad
```

1	1	1	1	2
2	2	2	1	5
3	3	3	2	1
4	4	1	2	1
5	1	2	1	3

2. LOAD - Cargar datos a tablas ELT

En ELT los datos se cargan sin transformar a las tablas destino. La limpieza vendrá después con UPDATE directamente sobre estas tablas.

Esta es la diferencia con ETL: primero cargamos, luego transformamos dentro de la base de datos.

```
In [17]: %%sql
CREATE TABLE elt_clientes AS
SELECT * FROM stg_clientes;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

```
Out[17]: []
```

```
In [18]: %%sql
CREATE TABLE elt_productos AS
SELECT * FROM stg_productos;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

```
Out[18]: []
```

```
In [19]: %%sql
CREATE TABLE elt_empleados AS
SELECT * FROM stg_empleados;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

```
Out[19]: []
```

```
In [20]: %%sql
-- La tabla de ventas se crea con columnas extra para precio y total
-- que se calcularán en la fase de Transform
CREATE TABLE elt_ventas (
    venta_id INT,
    cliente_id INT,
    producto_id INT,
    empleado_id INT,
    cantidad INT,
    precio DECIMAL(10,2),
    total DECIMAL(10,2)
);

INSERT INTO elt_ventas (venta_id, cliente_id, producto_id, empleado_id, cantidad)
SELECT venta_id, cliente_id, producto_id, empleado_id, cantidad
FROM stg_ventas;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
5 rows affected.

Out[20]: []

Verificar carga inicial

```
In [21]: %%sql
-- Las tablas ELT tienen los mismos datos crudos (con NULLs y duplicados)
SELECT * FROM elt_clientes;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

Out[21]:

cliente_id	nombre	ciudad	edad
1	Carlos	San Salvador	25
2	Ana	Santa Ana	30
3	None	San Miguel	22
4	Pedro	La Libertad	40
4	Pedro	La Libertad	40

```
In [22]: %%sql
-- precio y total aún son NULL, se calculan en Transform
SELECT * FROM elt_ventas;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

```
Out[22]:
```

venta_id	cliente_id	producto_id	empleado_id	cantidad	precio	total
1	1	1	1	2	None	None
2	2	2	1	5	None	None
3	3	3	2	1	None	None
4	4	1	2	1	None	None
5	1	2	1	3	None	None

3. TRANSFORM - Limpieza y enriquecimiento dentro de la BD

Las transformaciones se aplican **con UPDATE y recreación de tablas directamente** sobre los datos ya cargados.

Problema	Tabla	Solución SQL
nombres NULL en clientes	elt_clientes	UPDATE ... SET nombre = 'Sin Nombre'
Filas duplicadas en clientes	elt_clientes	DISTINCT + DROP + RENAME
producto NULL	elt_productos	UPDATE ... SET producto = 'Producto Desconocido'
nombre NULL en empleados	elt_empleados	UPDATE ... SET nombre = 'Sin Nombre'
precio y total vacíos	elt_ventas	UPDATE + JOIN con elt_productos

Limpiar nombres NULL en clientes

```
In [23]: %%sql
UPDATE elt_clientes
SET nombre = 'Sin Nombre'
WHERE nombre IS NULL;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
1 rows affected.

Out[23]: []
```

Eliminar duplicados en clientes

```
In [24]: %%sql
-- Crear tabla temporal sin duplicados
CREATE TABLE clientes_temp AS
SELECT DISTINCT *
FROM elt_clientes;
```

```
-- Reemplazar la tabla original
DROP TABLE elt_clientes;
RENAME TABLE clientes_temp TO elt_clientes;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
0 rows affected.
0 rows affected.
```

Out[24]: []

```
In [25]: %%sql
-- Antes: 5 filas con duplicado y NULL | Después: 4 filas limpias
SELECT * FROM elt_clientes;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[25]:

cliente_id	nombre	ciudad	edad
1	Carlos	San Salvador	25
2	Ana	Santa Ana	30
3	Sin Nombre	San Miguel	22
4	Pedro	La Libertad	40

Limpiar nombres NULL en productos

```
In [26]: %%sql
UPDATE elt_productos
SET producto = 'Producto Desconocido'
WHERE producto IS NULL;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
1 rows affected.
```

Out[26]: []

```
In [27]: %%sql
SELECT * FROM elt_productos;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[27]:

producto_id	producto	categoria	precio
1	Laptop	Tecnologia	1200.00
2	Mouse	Tecnologia	25.00
3	Silla	Muebles	150.00
4	Producto Desconocido	Tecnologia	75.00

Limpiar nombres NULL en empleados

```
In [28]: %%sql
UPDATE elt_empleados
SET nombre = 'Sin Nombre'
WHERE nombre IS NULL;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
1 rows affected.
```

Out[28]: []

```
In [29]: %%sql
SELECT * FROM elt_empleados;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

Out[29]: **empleado_id nombre departamento**

1	Maria	Ventas
2	Jose	Ventas
3	Sin Nombre	Marketing

Calcular métricas de ventas (precio y total)

```
In [30]: %%sql
UPDATE elt_ventas v
JOIN elt_productos p
ON v.producto_id = p.producto_id
SET
v.precio = p.precio,
v.total = v.cantidad * p.precio;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

Out[30]: []

```
In [31]: %%sql
-- precio y total ahora están calculados
SELECT * FROM elt_ventas;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

Out[31]: **venta_id cliente_id producto_id empleado_id cantidad precio total**

1	1	1	1	2	1200.00	2400.00
2	2	2	1	5	25.00	125.00
3	3	3	2	1	150.00	150.00
4	4	1	2	1	1200.00	1200.00
5	1	2	1	3	25.00	75.00

4. NORMALIZACIÓN - Crear sub-dimensiones

Esta es la característica clave del Snowflake Schema: las dimensiones se normalizan extrayendo atributos repetidos a tablas separadas.

Sub-dimensión	Extraída de	Propósito
dim_ciudad	elt_clientes.ciudad	Evitar repetir el nombre de ciudad en cada cliente
dim_categoria	elt_productos.categoria	Evitar repetir la categoría en cada producto
dim_departamento	elt_empleados.departamento	Evitar repetir el departamento en cada empleado

dim_ciudad

```
In [32]: %%sql
CREATE TABLE dim_ciudad (
    ciudad_id INT AUTO_INCREMENT PRIMARY KEY,
    ciudad VARCHAR(100) UNIQUE
);

INSERT INTO dim_ciudad (ciudad)
SELECT DISTINCT ciudad
FROM elt_clientes;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
4 rows affected.

Out[32]: []

```
In [33]: %%sql
SELECT * FROM dim_ciudad;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.

Out[33]:

ciudad_id	ciudad
4	La Libertad
3	San Miguel
1	San Salvador
2	Santa Ana

dim_categoria

```
In [34]: %%sql
CREATE TABLE dim_categoria (
```

```

        categoria_id INT AUTO_INCREMENT PRIMARY KEY,
        categoria    VARCHAR(100) UNIQUE
    );

INSERT INTO dim_categoria (categoria)
SELECT DISTINCT categoria
FROM elt_productos;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
2 rows affected.

```

Out[34]: []

```

In [35]: %%sql
SELECT * FROM dim_categoria;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
2 rows affected.

```

Out[35]: **categoria_id** **categoria**

2	Muebles
---	---------

1	Tecnologia
---	------------

dim_departamento

```

In [36]: %%sql
CREATE TABLE dim_departamento (
    departamento_id INT AUTO_INCREMENT PRIMARY KEY,
    departamento    VARCHAR(100) UNIQUE
);

INSERT INTO dim_departamento (departamento)
SELECT DISTINCT departamento
FROM elt_empleados;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
2 rows affected.

```

Out[36]: []

```

In [37]: %%sql
SELECT * FROM dim_departamento;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
2 rows affected.

```

Out[37]: **departamento_id** **departamento**

2	Marketing
---	-----------

1	Ventas
---	--------

5. CREACIÓN DE DIMENSIONES - Con FK a sub-dimensiones

Las dimensiones principales ahora referencian a sus sub-dimensiones mediante FOREIGN KEY. Esto elimina la redundancia: en lugar de repetir 'San Salvador' en cada cliente, solo se guarda el `ciudad_id`.

dim_cliente → referencia dim_ciudad

```
In [38]: %%sql
CREATE TABLE dim_cliente (
    cliente_id INT PRIMARY KEY,
    nombre     VARCHAR(100),
    edad       INT,
    ciudad_id  INT,

    FOREIGN KEY (ciudad_id)
        REFERENCES dim_ciudad(ciudad_id)
);

INSERT INTO dim_cliente (cliente_id, nombre, edad, ciudad_id)
SELECT
    c.cliente_id,
    c.nombre,
    c.edad,
    ci.ciudad_id
FROM elt_clientes c
JOIN dim_ciudad ci
    ON c.ciudad = ci.ciudad;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
4 rows affected.
```

Out[38]: []

```
In [39]: %%sql
SELECT * FROM dim_cliente;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[39]: **cliente_id nombre edad ciudad_id**

1	Carlos	25	1
2	Ana	30	2
3	Sin Nombre	22	3
4	Pedro	40	4

dim_producto → referencia dim_categoria

```
In [40]: %%sql
CREATE TABLE dim_producto (
    producto_id INT PRIMARY KEY,
    producto VARCHAR(100),
    precio DECIMAL(10,2),
    categoria_id INT,

    FOREIGN KEY (categoria_id)
        REFERENCES dim_categoria(categoria_id)
);

INSERT INTO dim_producto (producto_id, producto, precio, categoria_id)
SELECT
    p.producto_id,
    p.producto,
    p.precio,
    c.categoria_id
FROM elt_productos p
JOIN dim_categoria c
ON p.categoria = c.categoria;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
4 rows affected.
```

Out[40]: []

```
In [41]: %%sql
SELECT * FROM dim_producto;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[41]:

producto_id	producto	precio	categoria_id
1	Laptop	1200.00	1
2	Mouse	25.00	1
3	Silla	150.00	2
4	Producto Desconocido	75.00	1

dim_empleado → referencia dim_departamento

```
In [42]: %%sql
CREATE TABLE dim_empleado (
    empleado_id INT PRIMARY KEY,
    nombre VARCHAR(100),
    departamento_id INT,

    FOREIGN KEY (departamento_id)
        REFERENCES dim_departamento(departamento_id)
);

INSERT INTO dim_empleado (empleado_id, nombre, departamento_id)
```

```
SELECT
    e.empleado_id,
    e.nombre,
    d.departamento_id
FROM elt_empleados e
JOIN dim_departamento d
    ON e.departamento = d.departamento;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
3 rows affected.
```

Out[42]: []

```
In [43]: %%sql
SELECT * FROM dim_empleado;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

Out[43]: **empleado_id nombre departamento_id**

1	Maria	1
2	Jose	1
3	Sin Nombre	2

6. TABLA DE HECHOS - fact_ventas

La tabla de hechos es el centro del Snowflake Schema. Contiene las métricas medibles y las **Foreign Keys** que apuntan a las dimensiones principales.

Nota: fact_ventas no apunta directamente a dim_ciudad, dim_categoria ni dim_departamento. Para llegar a ellas se llega a través de las dimensiones principales.

```
In [44]: %%sql
CREATE TABLE fact_ventas (
    venta_id      INT PRIMARY KEY,
    cliente_id    INT,
    producto_id   INT,
    empleado_id   INT,
    cantidad      INT,
    precio        DECIMAL(10,2),
    total         DECIMAL(10,2),

    FOREIGN KEY (cliente_id) REFERENCES dim_cliente(cliente_id),
    FOREIGN KEY (producto_id) REFERENCES dim_producto(producto_id),
    FOREIGN KEY (empleado_id) REFERENCES dim_empleado(empleado_id)
);
```

```
INSERT INTO fact_ventas
SELECT * FROM elt_ventas;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
5 rows affected.
```

Out[44]: []

```
In [45]: %%sql
SELECT * FROM fact_ventas;
```

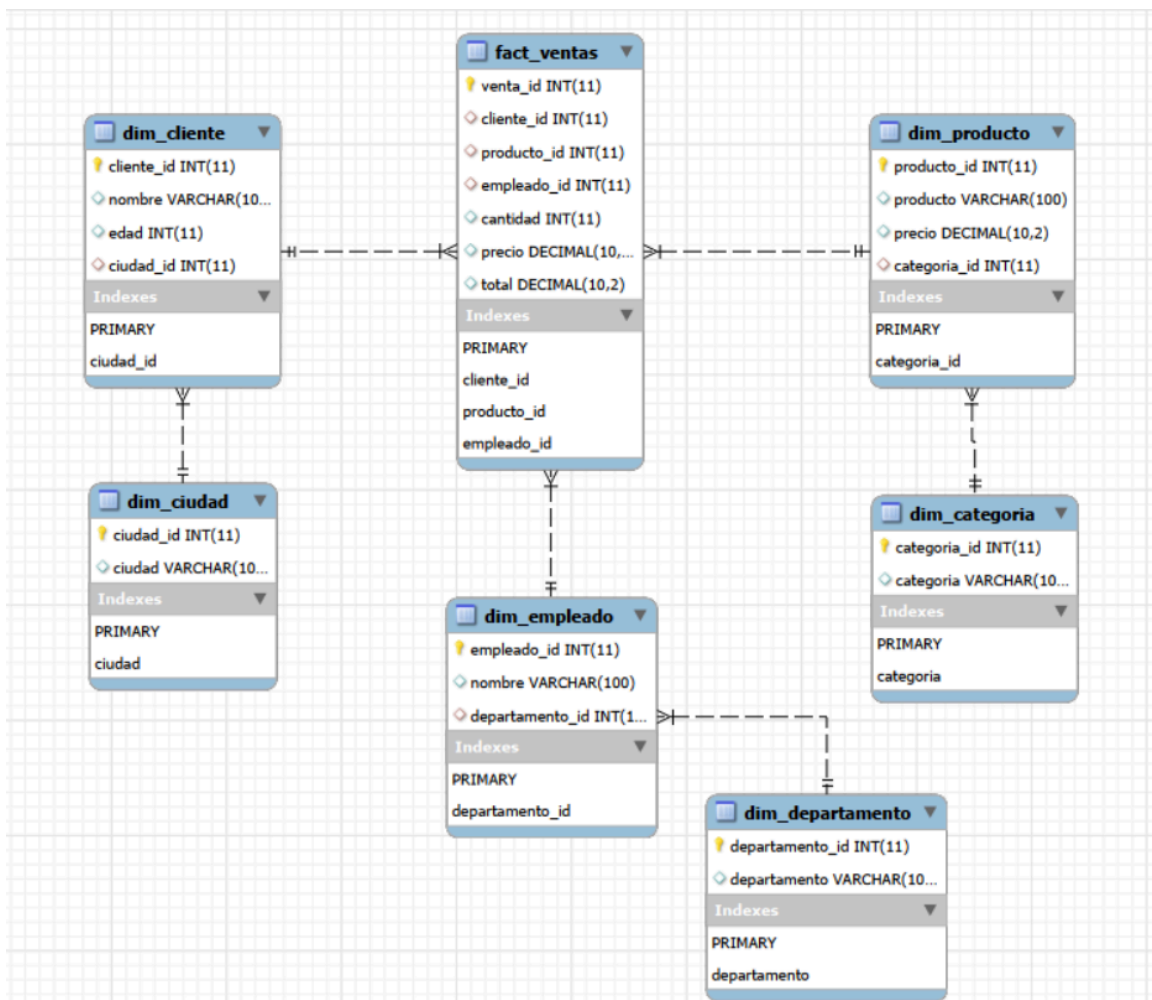
```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

Out[45]:

venta_id	cliente_id	producto_id	empleado_id	cantidad	precio	total
----------	------------	-------------	-------------	----------	--------	-------

1	1	1	1	2	1200.00	2400.00
2	2	2	1	5	25.00	125.00
3	3	3	2	1	150.00	150.00
4	4	1	2	1	1200.00	1200.00
5	1	2	1	3	25.00	75.00

7. Diagrama del Snowflake Schema



Tenemos 3 niveles de tablas:

fact_ventas: es la tabla más importante. Guarda cada venta que ocurrió: cuánto se vendió, a qué precio y el total. No repite información solo guarda los IDs que apuntan a las demás tablas.

las dimensiones principales: son **dim_cliente**, **dim_producto** y **dim_empleado**. Describen quién compró, qué se vendió y quién vendió. Aquí está la diferencia con el Star Schema: estas tablas no guardan directamente la ciudad, la categoría o el departamento, solo guardan un ID que apunta al siguiente nivel.

las sub-dimensiones: son **dim_ciudad**, **dim_categoria** y **dim_departamento**. Aquí es donde vive el dato normalizado. Por ejemplo, "San Salvador" solo se escribe una vez en **dim_ciudad**, y todos los clientes de esa ciudad simplemente apuntan a ese registro para no repetir datos. Si mañana "San Salvador" cambia de nombre, solo se modifica en un lugar, no en cada fila de clientes. Eso se llama normalización, y es la característica clave del Snowflake Schema.

8. KPIs - Consultas analíticas

En el Snowflake Schema los JOINS son más largos porque hay que navegar por las sub-dimensiones para llegar a los atributos normalizados.

KPI 1 - Ingreso total y promedio por venta

```
In [46]: %%sql
SELECT
    COUNT(*) AS num_ventas,
    SUM(total) AS ingreso_total,
    AVG(total) AS promedio_venta
FROM fact_ventas;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
1 rows affected.

```
Out[46]: num_ventas ingreso_total promedio_venta
```

5	3950.00	790.000000
---	---------	------------

KPI 2 - Ventas por ciudad

En Snowflake Schema hay que hacer JOIN doble: fact_ventas → dim_cliente → dim_ciudad

```
In [47]: %%sql
SELECT
    ci.ciudad,
    SUM(f.total) AS ventas
FROM fact_ventas f
JOIN dim_cliente c
    ON f.cliente_id = c.cliente_id
JOIN dim_ciudad ci
    ON c.ciudad_id = ci.ciudad_id
GROUP BY ci.ciudad
ORDER BY ventas DESC;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.

```
Out[47]: ciudad ventas
```

San Salvador	2475.00
La Libertad	1200.00
San Miguel	150.00
Santa Ana	125.00

KPI 3 - Ingresos por categoría de producto

JOIN doble: fact_ventas → dim_producto → dim_categoria

```
In [48]: %%sql
SELECT
    cat.categoria,
    SUM(f.total) AS ingresos
FROM fact_ventas f
JOIN dim_producto p
    ON f.producto_id = p.producto_id
JOIN dim_categoria cat
    ON p.categoria_id = cat.categoria_id
GROUP BY cat.categoria
ORDER BY ingresos DESC;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
2 rows affected.
```

```
Out[48]: categoria  ingresos
Tecnologia      3800.00
Muebles         150.00
```

KPI 4 - Rendimiento de empleados por departamento

JOIN doble: fact_ventas → dim_empleado → dim_departamento

```
In [49]: %%sql
SELECT
    e.nombre      AS empleado,
    d.departamento,
    SUM(f.total)  AS ventas_generadas
FROM fact_ventas f
JOIN dim_empleado e
    ON f.empleado_id = e.empleado_id
JOIN dim_departamento d
    ON e.departamento_id = d.departamento_id
GROUP BY e.nombre, d.departamento
ORDER BY ventas_generadas DESC;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
2 rows affected.
```

```
Out[49]: empleado  departamento  ventas_generadas
Maria      Ventas      2600.00
Jose       Ventas      1350.00
```

KPI 5 - Top clientes por monto comprado

```
In [50]: %%sql
SELECT
    c.nombre      AS cliente,
    ci.ciudad,
    SUM(f.total)  AS total_comprado
```

```

FROM fact_ventas f
JOIN dim_cliente c
  ON f.cliente_id = c.cliente_id
JOIN dim_ciudad ci
  ON c.ciudad_id = ci.ciudad_id
GROUP BY c.nombre, ci.ciudad
ORDER BY total_comprado DESC;

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/

4 rows affected.

Out[50]:

cliente	ciudad	total_comprado
Carlos	San Salvador	2475.00
Pedro	La Libertad	1200.00
Sin Nombre	San Miguel	150.00
Ana	Santa Ana	125.00

9. Resumen del proceso ELT + Snowflake Schema

Paso	Fase	Acción
1	Extract	Datos crudos cargados en tablas staging (con NULLs y duplicados)
2	Load	Copia directa de staging a tablas elt_ sin transformar
3	Transform	UPDATE para NULLs, DISTINCT+RENAME para duplicados, JOIN para calcular total
4	Normalización	Sub-dimensiones: dim_ciudad, dim_categoria, dim_departamento
5	Dimensiones	dim_cliente, dim_producto, dim_empleado con FK a sub-dimensiones
6	Fact Table	fact_ventas con FK a dimensiones principales y métricas de negocio
7	KPIs	5 consultas analíticas con gráficos y JOINS por sub-dimensiones

Problemas detectados y resueltos

Problema	Tabla	Solución
Fila duplicada Pedro id=4	stg_clientes	SELECT DISTINCT + DROP + RENAME
nombre = NULL	stg_clientes, stg_empleados	UPDATE ... SET nombre = 'Sin Nombre'
producto = NULL	stg_productos	UPDATE ... SET producto = 'Producto Desconocido'
precio y total vacíos	elt_ventas	UPDATE + JOIN con elt_productos
ciudad repetida en clientes	elt_clientes	Normalizada en dim_ciudad

Problema	Tabla	Solución
categoría repetida en productos	elt_productos	Normalizada en dim_categoria
departamento repetido en empleados	elt_empleados	Normalizado en dim_departamento

ETL vs ELT - ¿Cuál usar?

Criterio	ETL	ELT
¿Dónde se transforma?	Antes de cargar (fuera de BD)	Después de cargar (dentro de BD)
Ideal para	Datos sensibles, recursos limitados	Big Data, bases de datos potentes
Velocidad de carga	Más lenta	Más rápida
Herramientas	Python, Spark	SQL, dbt, Snowflake